Name: Muhammad Noor
Roll No: i232520
Section: DS–A
Submission Date: *November 2, 2025*
Assignment: 2

Abstract

This report presents the design and implementation of an **Intelligent CV Analyzer** that leverages classical *string matching algorithms* — Brute Force, Rabin–Karp, and Knuth–Morris–Pratt (KMP) — to automate the process of extracting skills and evaluating job–fit relevance from candidate resumes. The system aims to reduce manual screening time by accurately matching candidate skills with job description keywords. It further compares algorithm performance based on execution time and character comparisons across varying CV sizes and keyword workloads. Experimental results demonstrate that the KMP algorithm performs most efficiently on large text datasets, providing a balance between accuracy and speed.

Contents

1 Introduction and Problem Definition **3**

2 Objectives **3**

3 System Design **3**
3.1 Architecture Overview 3
3.2 System Flow 3
3.3 Flowchart 4

4 Algorithm Explanation and Pseudocode **5**
4.1 1. Brute Force Search 5
4.2 2. Rabin–Karp Algorithm 5
4.3 3. Knuth–Morris–Pratt (KMP) Algorithm 6

5 Implementation Details **6**

6 Application Interface and Visualization Preview **7**
6.1 Application Screenshot 7

7 Experimental Results and Analysis **7**
7.1 Performance Metrics 7
7.2 Results Visualization 8
7.3 Comparative Analysis 13
7.4 Trade–off Discussion 14

8 Conclusion and Future Improvements **14**

9 References **14**

List of Figures

1 Flowchart of Intelligent CV Analyzer System 4
2 Main Application Interface 7
3 Uploading CV And Job Description 8
4 Analysis Results - Ranked Candidates 9
5 Algorithm Rankings 10
6 Performance Metrics 10
7 Scenario Comparison 11
8 CV Size And Keyword Mode Breakdown 12
9 Detailed Performance Table And Algorithm Analysis 13
10 Recommendation Insights 13

1 Introduction and Problem Definition

The recruitment process often involves screening hundreds of CVs for each job position. Manually identifying relevant skills and qualifications is both time–consuming and error–prone due to the unstructured nature of resumes. This project aims to design a web–based system capable of **automatically extracting** keywords from CVs and matching them with predefined job descriptions using efficient string matching algorithms.

The problem statement focuses on automating skill extraction and candidate ranking through the implementation of three core algorithms: *Brute Force*, *Rabin–Karp*, and *Knuth–Morris–Pratt (KMP)*.

2 Objectives

- Implement and compare Brute Force, Rabin–Karp, and KMP algorithms for text analysis.
- Develop a web application capable of analyzing CVs in PDF, DOCX, or TXT format.
- Extract and match skills, tools, and technologies from CV text.
- Compare candidate profiles with job description keywords.
- Analyze the efficiency, accuracy, and performance of each algorithm.
- Recommend the most suitable algorithm for real–time CV screening.

3 System Design

3.1 Architecture Overview

The system follows a client–server model implemented using the **Flask** framework. The backend handles file uploads, text extraction, algorithm processing, and performance computation, while the frontend provides an interactive web interface for users to upload CVs and select job templates.

3.2 System Flow

1. User uploads a CV file and selects a job template.
2. The application extracts text content (via PyPDF2 or python–docx).
3. Extracted text is processed using Brute Force, Rabin–Karp, and KMP algorithms.
4. Keywords from the job description are matched against CV text.
5. Scores, matches, and performance metrics are calculated and displayed.

3.3 Flowchart

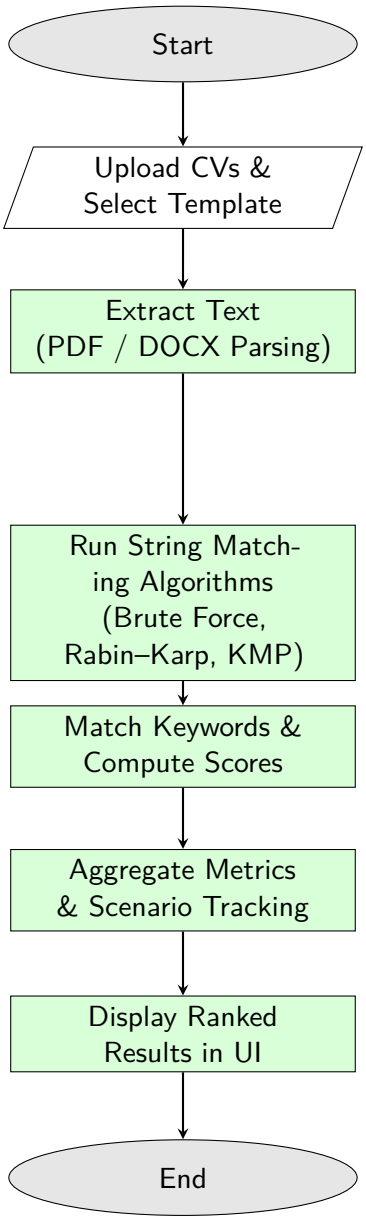

Figure 1: Flowchart of Intelligent CV Analyzer System

4 Algorithm Explanation and Pseudocode

This section presents the detailed explanation and pseudocode of the three string matching algorithms implemented in the CV Analyzer: **Brute Force Search**, **Rabin–Karp**, and **Knuth–Morris–Pratt (KMP)**. Each algorithm is evaluated based on execution time, number of comparisons, and efficiency in handling different CV sizes and keyword workloads.

4.1 1. Brute Force Search

Explanation:
It slides the pattern over the text one character at a time, comparing characters sequentially. Every mismatch triggers a new alignment of the pattern. It counts all comparisons and records match indices.

Pseudocode:

```
BRUTE_FORCE_SEARCH(text, pattern)
    If case-insensitive: convert both text and pattern to lowercase
    Initialize n ← length(text), m ← length(pattern)
    Initialize matches ← empty list, comparisons ← 0
    Start timer
    For i ← 0 to n - m:
      j ← 0
      While j < m and text[i + j] == pattern[j]:
        comparisons ← comparisons + 1
        j ← j + 1
      comparisons ← comparisons + 1 // failed comparison
      If j == m:
        Append i to matches
    Stop timer and compute execution_time
    Return matches, count, comparisons, execution_time
```

4.2 2. Rabin–Karp Algorithm

Explanation:
It uses a rolling hash to speed up pattern matching. It computes hash values for the pattern and each substring of the text, comparing hashes first and only checking characters if hashes match (reducing unnecessary comparisons).

Pseudocode:

```
RABIN_KARP_SEARCH(text, pattern, base=256, prime=101)
    If case-insensitive: convert both text and pattern to lowercase
    Initialize n ← len(text), m ← len(pattern)
    If n < m: return no matches
    Initialize pattern_hash, text_hash, h ← 0, 0, 1
    For i ← 0 to m - 2:
      h ← (h * base) mod prime
    For i ← 0 to m - 1:
      pattern_hash ← (base * pattern_hash + ord(pattern[i])) mod prime
      text_hash ← (base * text_hash + ord(text[i])) mod prime
    Start timer
    For i ← 0 to n - m:
      If pattern_hash == text_hash:
        Compare characters one by one
```

```
        If all match: record match index
    If i < n - m:
        text_hash ← (base * (text_hash - ord(text[i]) * h) + ord(text[i + m])) mod prime
        If text_hash < 0: text_hash ← text_hash + prime
Stop timer and compute execution_time
Return matches, count, comparisons, execution_time
```

4.3 3. Knuth–Morris–Pratt (KMP) Algorithm

Explanation:
It improves search efficiency by preprocessing the pattern to build an LPS (Longest Prefix Suffix) array. This allows the algorithm to skip unnecessary character comparisons when mismatches occur.

Pseudocode:

```
KMP_SEARCH(text, pattern)
If case-insensitive: convert both text and pattern to lowercase
Initialize n ← len(text), m ← len(pattern)
Build LPS array:
    lps[0] ← 0, length ← 0
    For i ← 1 to m - 1:
        If pattern[i] == pattern[length]:
            length ← length + 1
            lps[i] ← length
        Else if length != 0:
            length ← lps[length - 1]
        Else:
            lps[i] ← 0
Initialize i ← 0 (text index), j ← 0 (pattern index)
Start timer
While i < n:
    comparisons ← comparisons + 1
    If pattern[j] == text[i]:
        i ← i + 1, j ← j + 1
    If j == m:
        Record match (i - j)
        j ← lps[j - 1]
    Else if i < n and pattern[j] != text[i]:
        If j != 0: j ← lps[j - 1]
        Else: i ← i + 1
Stop timer and compute execution_time
Return matches, count, comparisons, execution_time
```

5 Implementation Details

Technologies Used:

- **Backend:** Flask 3.0.0 (Python)
- **Libraries:** PyPDF2, python–docx, Werkzeug
- **Frontend:** HTML5, CSS3, JavaScript
- **Visualization:** Chart.js

6 Application Interface and Visualization Preview

This section provides a visual overview of the Intelligent CV Analyzer web interface, illustrating how users interact with the system and how algorithmic results are displayed.

6.1 Application Screenshot

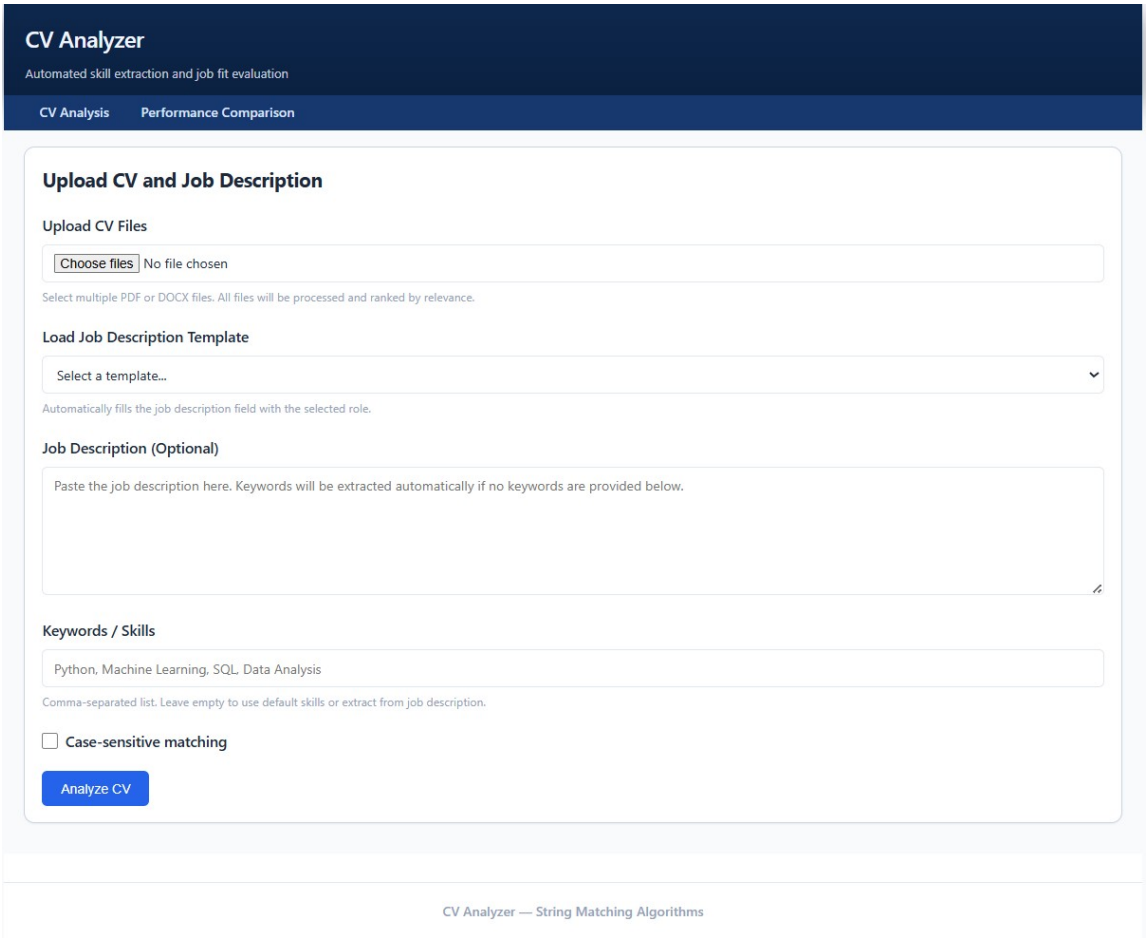

Figure 2: Main Application Interface

7 Experimental Results and Analysis

7.1 Performance Metrics

The system measures:

- Execution time (ms)
- Number of character comparisons
- Average time per CV and keyword mode

7.2 Results Visualization

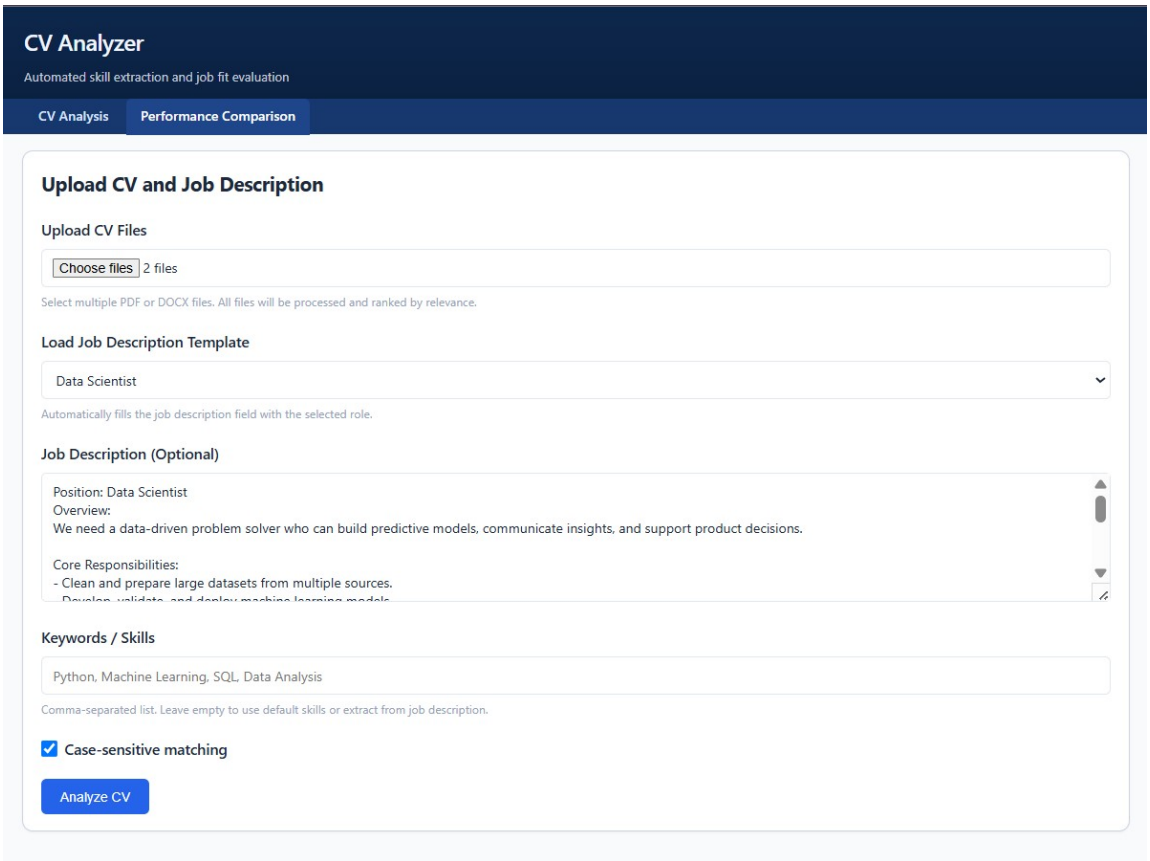

Figure 3: Uploading CV And Job Description

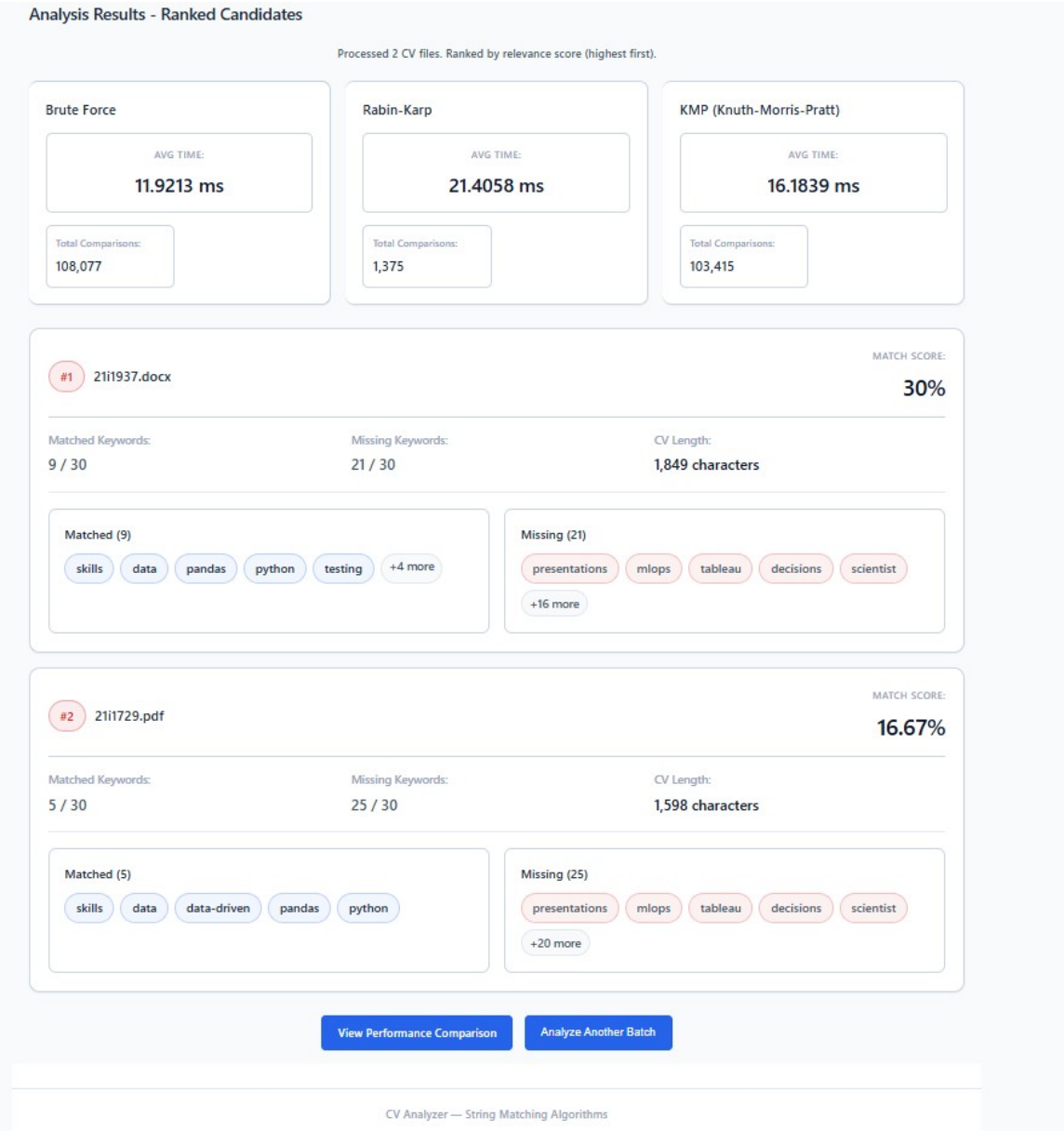

Figure 4: Analysis Results - Ranked Candidates

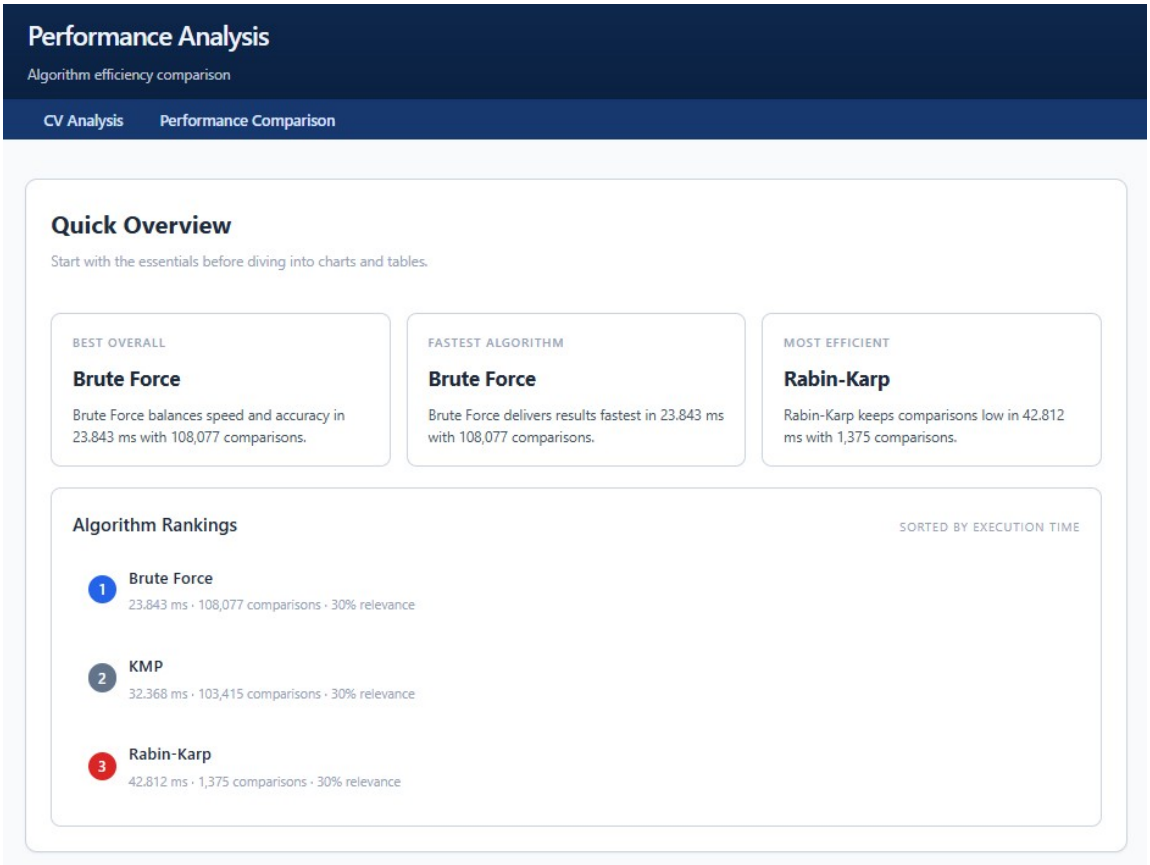

Figure 5: Algorithm Rankings

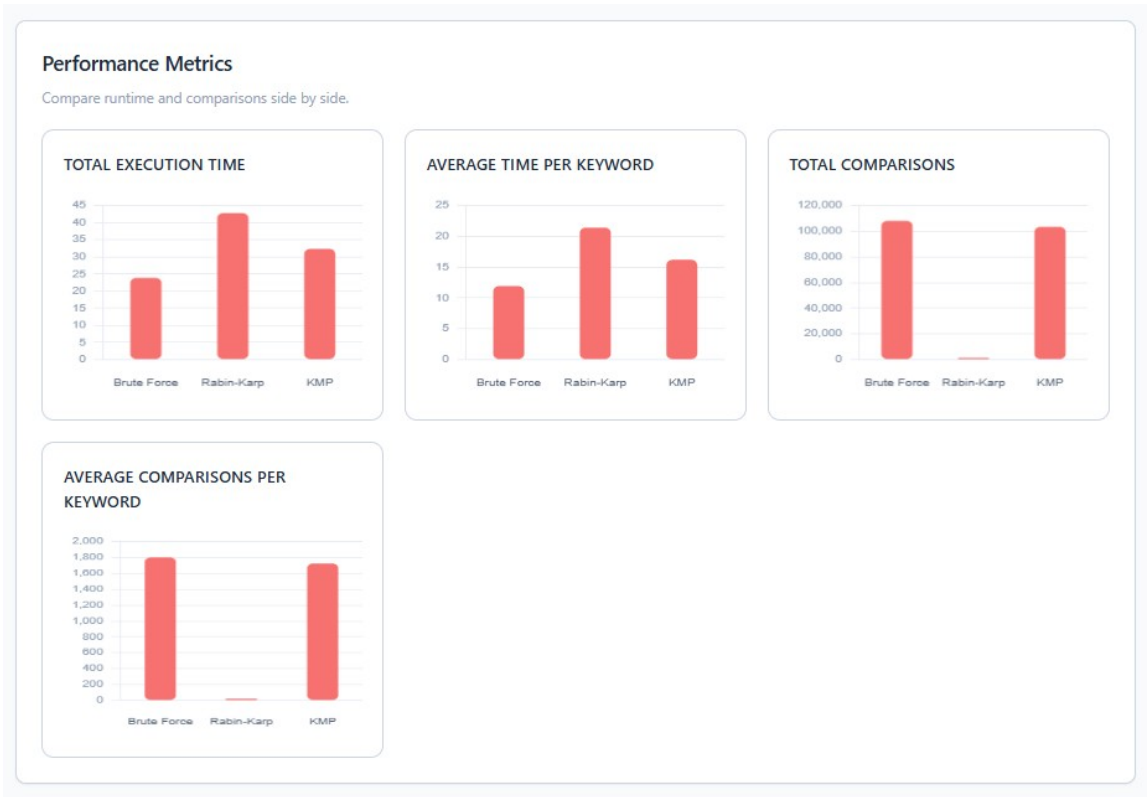

Figure 6: Performance Metrics

Figure 7: Scenario Comparison

CV Size Breakdown

SCENARIO	ALGORITHM	AVG TIME / CV (MS)	AVG TIME / KEYWORD (MS)	AVG COMPARISONS / CV	AVG COMPARISONS / KEYWORD	SAMPLES
Small CVs	**Brute Force**	11.9213	0.3974	54038.50	1801.28	2
Small CVs	**Rabin-Karp**	21.4058	0.7135	687.50	22.92	2
Small CVs	**KMP**	16.1839	0.5395	51707.50	1723.58	2

CVs with ≤ 5,000 characters are classified as small.

Keyword Mode Breakdown

SCENARIO	ALGORITHM	AVG TIME / CV (MS)	AVG TIME / KEYWORD (MS)	AVG COMPARISONS / CV	AVG COMPARISONS / KEYWORD	SAMPLES
Single Keyword	**Brute Force**	0.3724	0.3724	1764.50	1764.50	2
Single Keyword	**Rabin-Karp**	0.6771	0.6771	15.50	15.50	2
Single Keyword	**KMP**	0.5347	0.5347	1723.50	1723.50	2
Multiple Keywords	**Brute Force**	11.9213	0.3974	54038.50	1801.28	2
Multiple Keywords	**Rabin-Karp**	21.4058	0.7135	687.50	22.92	2
Multiple Keywords	**KMP**	16.1839	0.5395	51707.50	1723.58	2

Single keyword results use the first keyword supplied in the form.

Figure 8: CV Size And Keyword Mode Breakdown

Detailed Performance Table

Hover a row to highlight a single algorithm's metrics.

ALGORITHM	RELEVANCE SCORE	TOTAL TIME (MS)	AVG TIME (MS)	TOTAL COMPARISONS	AVG COMPARISONS	MATCHED KEYWORDS	MISSING KEYWORDS
Brute Force	30%	23.8425	11.9213	108,077	1801.28	9	21
Rabin-Karp	30%	42.8116	21.4058	1,375	22.92	9	21
KMP	30%	32.3679	16.1839	103,415	1723.58	9	21

Algorithm Analysis

Reference time and space complexity alongside practical trade-offs.

ALGORITHM	TIME COMPLEXITY	SPACE COMPLEXITY	ADVANTAGES	DISADVANTAGES
Brute Force	O(n*m) n = text length, m = pattern length	O(1)	Simple implementation, no preprocessing required	Less efficient for large texts, performs redundant comparisons
Rabin-Karp	O(n+m) avg O(n*m) worst case	O(1)	Good average-case performance, efficient for multiple patterns	Hash collisions can degrade performance, worst-case is slow
Knuth-Morris-Pratt (KMP)	O(n+m)	O(m)	Guaranteed linear time, no backtracking in text	Requires preprocessing (LPS array), additional space

Figure 9: Detailed Performance Table And Algorithm Analysis

Recommendation

Tailored guidance based on the current dataset.

Based on the performance analysis: **Brute Force** has the fastest execution time (23.8425 ms), while **Rabin-Karp** requires the fewest comparisons (1,375). For real-time CV screening with 30 keywords across 2 CVs (avg 925 characters), **Brute Force** is simple but may not scale well for large CVs or many keywords. Consider KMP or Rabin-Karp for production use.

Analyze Another CV

CV Analyzer — String Matching Algorithms

Figure 10: Recommendation Insights

7.3 Comparative Analysis

Algorithm	Strengths	Limitations
Brute Force	Simple and intuitive	Inefficient on large text datasets
Rabin–Karp	Fast on average cases	Hash collisions possible

KMP	Best overall time complexity	Slightly higher preprocessing time

7.4 Trade–off Discussion

The trade–off between *speed* and *accuracy* is evident:

- Brute Force ensures accuracy but lacks scalability.
- Rabin–Karp improves speed but suffers from hash collisions.
- KMP balances preprocessing with consistent linear–time performance.

8 Conclusion and Future Improvements

This project successfully demonstrates the application of classical string–matching algorithms in resume analysis. Among the tested methods, KMP provided the most consistent and efficient performance across varying CV sizes and keyword workloads.

Future Enhancements:

- Integrate NLP–based keyword weighting and semantic analysis.
- Expand dataset handling for multiple CV uploads at once.
- Include cloud deployment (e.g., AWS) for real–time screening.
- Add exportable PDF reports for recruiters.

9 References

- Flask Documentation. https://flask.palletsprojects.com
- Chart.js Official Documentation. https://www.chartjs.org